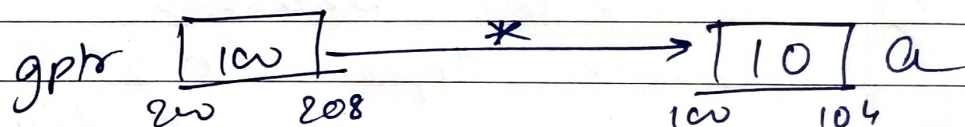


# ASSIGNMENT - 9

Page No.

Date: / /

- 1] What is pointer? explain term with an example.
- 
- Derived datatype in C & C++
  - In Java - Not applicable
  - Def<sup>n</sup>:
- 1) Variable which store memory address
  - 2) special variable which can store memory address of anything.
- Power - To fetch data whose address is stored in it.
- we can create a pointer, which points to any primitive data type, array, struct and so on.
  - Address held by pointer → 'unsigned long'
- ∴ size of every pointer = 8 byte.
- Eg `int *gptr = &a;`
- Here, `gptr` is a pointer pointing to integer datatype and holds address of `a`.



- 2] What is mean by Null pointer? Why we must initialize the pointer to NULL?
- 
- 1) If we create any pointer & it is not initialized it may store garbage value in it.
  - 2) Due to this unwanted garbage value it may lead to - 'Runtime failure'
  - 3) To avoid this, we use concept of NULL ptr. NULL - macro defined `stdio.h` header file. value = 0 (zero)
  - 4) when we initialize any uninitialized pointer with NULL it helps to reduce the runtime accidents (Segmentation fault).



- 5) Recommended : Initialize pointer with NULL rather than keeping it empty in order to avoid runtime failure.  
Ex. `int *nptr = NULL;`
- 5) What are different types of pointers & list out them.
- 1) Integer Pointers :  
`int *ptr = &a;` (Pointer pointing to integer datatype).
- 2) Array pointer :  
`int Arr = {10, 20, 30};`  
`int *ptr = &Arr;`  
 Pointer pointing to an array.
- 3) Function pointer :  
`int (*ptr) (int, char);`  
 - Points to a function instead of data & is used to call function dynamically.
- 4) Double Pointers (Pointer to Pointer)  
`datatype **pointer-name;`  
 A pointer pointing another pointer.
- 5) NULL Pointer :  
 They don't point to any memory location.  
`datatype *pointname = NULL;`
- 6) void Pointer :  
 Called → Generic Pointer  
 [Can point to any types & typecasted to any type]  
`void *ptr = &a;`
- 7) wild Pointers :  
`int *ptr;`  
 uninitialized pointers, → can cause problems & crash programs.



## 8] Structure Pointer :

struct struct-name \*ptr ;

## 9] Constant Pointer :

Memory address stored inside pointer is constant & can't be modified.

datatype \*const pointer\_name ;

## 10] Pointer to constant :

Pointer pointing to constant value, that can't be modified.

const - datatype \* pointer\_name ;

## 11] Other Types :

- |                 |                       |
|-----------------|-----------------------|
| b) Raw pointer  | b) Dangling pointer.  |
| c) Huge pointer | d) Complex pointer    |
| e) Near pointer | f) Normalized pointer |
| g) File Pointer |                       |

## 4] Write notes on pointer, size of pointer, datatype of pointer

> 1) Pointer is a Derived datatype in C & C++ it doesn't exist in JAVA.

### 2) Power of Pointer :

- Store memory address of variable, array, structure, etc it is pointing to.
- Retrieving out the value/data stored in that memory address.

### 3) Size of pointer :

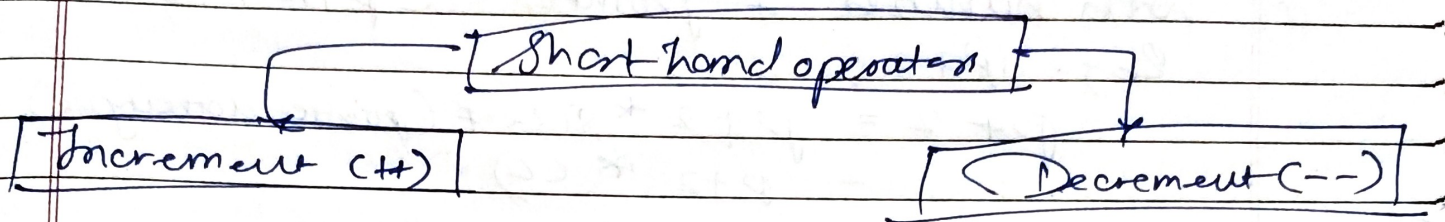
Pointer is a special variable which holds address, each address is of type - 'unsigned long'   
 Due to which - Size of every pointer = 8 byte



5) What is meant by increment & decrement operator?

→ 1) Iteration - Is a programming language paradigm, when we want to execute more than 1 statements multiple times.

2) In C, C++ & Java, there are 2 operators which are used to increment & decrement value by 1 respectively -



Pre increment

Post increment

Pre decrement

Post Decrement

No = ++1

No = 1++

No = --1

No = 1--

- All above operators are considered shorthand operators.

- Suppose No is a variable, which is of type integer & initialized with value 50

```
int No = 50;
```

No++; // just increment [Not pre/post]

- The above statements gets internally converted as  $\rightarrow No = No + 1;$

- The same thing happens with decrement operator

```
No--;
```

It gets converted into  $\rightarrow No = No - 1;$

\* Compiler will convert shorthand operators to normal statements.



77) Write a note on pointer arithmetic, explain in detail with example.

- 4 operations -  $+$ ,  $-$ ,  $\times$ ,  $\div$ .
- At least 2 pointers, same datatype and pointing to same memory region.
- Multiplication & division is not allowed.

### 1) Addition

a) with number + pointer [PTR + No]  
Ex.  $P+2$ ;

$$\begin{aligned} P+2 &= P+2 * \text{sizeof}(\text{pointer datatype}) \\ &= P+2 * (4) \\ &= 100+8 = \underline{108} \end{aligned}$$

b) pointer + pointer // Not Allowed

Ex.  $P+Q = 100 + 116 = 216$

→ If we do it, it is considered as encroachment to other space, so OS will kill the process.

### 2) Subtraction

a) Number (-) Pointer

Ex.  $P-2$

$$P-2 = 116 - 2 * (4) = \underline{108}$$

b) Pointer (-) Pointer

Ex.  $Q-P = 116 - 100 / 4 = 16/4 = 4$

### Increment Operator

$$\begin{aligned} P++ &= P+1 \\ &= P+1 * (4) \\ &= 100+4 \\ &= \underline{104} \end{aligned}$$

### Decrement Operator

$$\begin{aligned} Q-- &= Q-1 \\ &= Q-1 * (4) \\ &= 116-4 \\ &= \underline{112} \end{aligned}$$



- 6/ What is pointer to pointer, explain with example & write a program to demonstrate this.
- A pointer stores address of another pointer.  
 - used to indirectly manipulate or access the value of variable pointed to by the 1<sup>st</sup> pointer.

eg. `int i = 10;`  
`int *ptr1 = &i;`  
`int **ptr2 = &ptr1;`

3/ Code:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int no = 21;
```

```
    int *p = &no;
```

```
    int **a = &p;
```

```
    int ***q = &a;
```

```
    int ****r = &q;
```

```
    int *****x = &r;
```

```
    int ****y = &x;
```

```
    printf("value of *****y is %d\n", &*****y);
```

```
    return 0;
```

```
}
```

Output:

value of \*\*\*\*\*y is : 21



8] Explain how array is considered as pointer & pointer can be treated as array?

- 1) Array is internally considered as a pointer.  
 2) As a programmer, when we access any element of array, the compiler will, internally convert ~~pointer~~ into its corresponding pointer's representation.

3) The concept of array & pointer & its correlation is same in C & C++ programming.

Eg. `int Arr[5] = {10, 20, 30, 40, 50};`

|     |     |    |    |    |     |
|-----|-----|----|----|----|-----|
|     | 0   | 1  | 2  | 3  | 4   |
| Arr | 10  | 20 | 30 | 40 | 50  |
|     | 100 |    |    |    | 120 |

- When we consider name of Array, it will give the base of 1<sup>st</sup> element (0th index)
- When we use the & operator with name of array, it will give address of whole array.
- When we use +1 operator on array i.e. `(Arr+1)` it gets shifted by size of single element forward i.e. `int = 4 bytes`.
- When we use +1 operator with & operator `&Arr+1` then it will give the address which shifted by size of whole array.
- When we access any element of array using `[]` operator, it is internally converted to its corresponding pointer representation.

$$\text{Arr}[3] = \text{Arr} + 3 = 100 + 3$$

$$= 100 + 12 = 112 = 40$$

\* Internally `* (Arr+1)`, which gives value access 1<sup>st</sup> element value (0th index).

So, to avoid this, array index in any PC starts from zero.



9) Predict the output of below code:

```
int no = 10;
int *p = NULL;
p = &no;
printf("%d", no);
printf("%d", *p);
*p = 11;
printf("%d", no);
printf("%d", *p);
return;
```

3

→ 1)  $\text{printf}(\text{"\%d", no}); \Rightarrow 10$   
 $\text{printf}(\text{"\%d", *p}); \Rightarrow 100 \leftarrow$   
 $\text{printf}(\text{"\%d", no}); \Rightarrow 11$   
 $\text{printf}(\text{"\%d", *p}); \Rightarrow 100 \leftarrow$   
 Address assumed to be 100

10) Predict output of below code.

- 1)  $\text{printf}(\text{"\%d", a}); // 100$  (base address)
- 2)  $\text{printf}(\text{"\%d", b}); // 108$  (3rd element address)
- 3)  $\text{printf}(\text{"\%d", *b}); // 40.6$
- 4)  $\text{printf}(\text{"\%d", *a}); // 10.5$
- 5)  $\text{printf}(\text{"\%d", *(a+2)}); // 30.5$
- 6)  $\text{printf}(\text{"\%d", *(a+1)}); // 20.2$
- 7)  $\text{printf}(\text{"\%d", a[1]}); // 20.2$
- 8)  $\text{printf}(\text{"\%d", *(2+arr)}); // 30.5$
- 9)  $\text{printf}(\text{"\%d", 0[arr]}); // 10.5$
- 10)  $\text{printf}(\text{"\%d", b-a}); // 3.000000$
- 11)  $\text{printf}(\text{"\%d", *(b-2)}); // 20.2$